

ROS 2

Workflow and basic communication

Roberto Masocco

`roberto.masocco@uniroma2.it`

Tor Vergata University of Rome

Department of Civil Engineering and Computer Science Engineering

May 6, 2026



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

Recap

The **robotics industry** is evolving rapidly towards the best practices adopted in the **software industry**.

ROS 2 is an open-source **middleware** for the development of robotics software and **distributed control architectures**.

ROS 2 is currently based on the **DDS** and **Zenoh** middleware to implement the communication layer.

Today, it is the **de facto standard** for the development of robotic applications, and it is supported by a **vast community** of developers and researchers.

This lecture is [here](#).

Roadmap

1 ROS 2 development workflow

2 Message topics

3 Hello, world!

Roadmap

1 ROS 2 development workflow

2 Message topics

3 Hello, world!

Installing ROS 2

HOWTO

On **Ubuntu** systems, the easiest way to install ROS 2 Jazzy is through [Debian packages](#).

The installation steps can be summarized as follows:

- ➊ ensure that **locales** are properly configured;
- ➋ add and configure **apt repositories**;
- ➌ **install** packages.

We have a script for this: [bin/ros2_jazzy_install.sh](#).

Installing ROS 2

Sourcing the installation

After the installation is complete, in order to use **CLI tools** and have libraries available to **build packages**, you need to **source the installation**:

```
source /opt/ros/jazzy/setup.bash (there are also a .zsh and a .sh)
```

so that your shell, and all its child processes from then on, will know the **paths** of all the **executables**, **shared objects** (libraries), and **include directories** installed by ROS 2, plus many **environment variables**.

Additional commands are required to set up **command line completion** and other useful environment variables.

We have a script for this too: [config/ros2_cmds.sh](#).

Source it, then you're good to go!

Language support

Low- vs high-level programming

Currently, ROS 2 officially supports **two programming languages**.

- **C++** (C++17 in Jazzy)
- **Python** (≥ 3.5 , 3.12 works with Jazzy)

As a standard practice, you may develop software packages using either one, but **each package** must be based on **only one** of them.

Language support

Low- vs high-level programming

The two languages are both fully supported since they are **complementary**.

- **C++** allows to build complex software using **modern paradigms**, but also to easily access the **hardware**, **libraries**, and **operating system** API when required, and to **optimize** the code for **performance**.
- **Python** allows to **rapidly prototype** software, especially high-level modules, to easily **interact** with the user, and to **visualize data** quickly; **AI** software modules are usually implemented in Python; the entire [ros2cli](#) suite is written in Python.

Note how one prioritizes other features with respect to the other, and vice versa.

This course will focus on C++ because of its major functionalities and widespread use in both the robotics industry and the development community.

The ROS 2 Python API is specular to the C++ one, easy to master after `rc1cpp`.

Language support

The build system

The ROS 2 [build system](#) supports both **C++** and **Python** packages through a common **package manager**: [colcon](#) (collective construction).

Spawned as a child project of the ROS community, its main features are:

- organization of the **build workspace** in a set of standard directories;
- **isolated** builds of packages, with **no pollution** of the system;
- automatic **dependency resolution** and **parallel** builds;
- support for **C/C++** packages through [CMake](#);
- support for **Python** packages through [setuptools](#).

Its configuration for a package can be found in the `package.xml` **manifest file**.

Language support

CMake

CMake is a **cross-platform build configuration generator**, which allows to build software using a **single, unified syntax** on all supported platforms.

Remember **Makefiles**? CMake is a compiler-agnostic **Makefile generator**.

It works with `CMakeLists.txt` files, which are essentially **scripts** that tell CMake how to build our software. The language may seem daunting but it's mostly boilerplate.

ROS 2 extends CMake with the [ament](#) library.

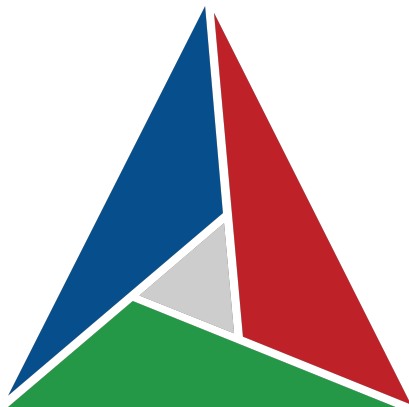


Figure 1: CMake logo.

Language support

setuptools

setuptools is a **Python library** designed to **standardize the setup of Python projects**, namely Python software packages.

Remember nightmarish import issues? setuptools is (also) a **dependency resolver**.

It works with

- `setup.py` files, in which the `setup(...)` function specifies the package's metadata and dependencies;
- `setup.cfg` files, which specify the location of all **executable scripts** in the package.



Figure 2: setuptools logo.

Language support

Building packages with colcon

The `colcon` command you will use the most is

```
colcon build
```

which builds all packages in the current workspace (*i.e.*, directory and child directories).

It has many options, but the most useful ones are:

- `--event-handlers` to specify which **build events** to log (e.g., `console_direct+`);
- `--packages-select` to specify which packages to build;
- `--symlink-install` to **symlink** executables and scripts into the `install/` directory, instead of copying them (useful for Python development: modifying a source file does not require a rebuild);
- `--packages-up-to` to build a package and all its dependencies;
- `--packages-ignore` to ignore a package and all its dependencies.

Language support

A note

Beware!

During development, a good 80% of all issues happens during integration and build.

We will see later on tools that help with this...

The workspace

Anatomy of a ROS 2 development directory

The organization of directories in a ROS 2 workspace is **standardized** because of colcon. We have, at least:

- build/ (autogenerated), which contains the **build artifacts** of all packages;
- install/ (autogenerated), which contains the **build products**;
- log/ (autogenerated), which contains the **build logs**;
- src/, which contains the **source code** of all packages.

If you use Git, remember to add build/, install/, and log/ to your .gitignore file!

Similarly, colcon ignores them when recursively looking for packages to build.

The workspace

Package creation

In the beginning was

```
ros2 pkg create <package_name>
```

which has way too many options. The main ones are:

- `--destination-directory src/`
- `--build-type <build_type> (ament_cmake or ament_python)`
- `--dependencies <package_name> ...`

Most of this stuff can be specified afterwards, eventually modifying the `package.xml` file.

Roadmap

1 ROS 2 development workflow

2 Message topics

3 Hello, world!

ROS 2 messages

A message is a **single data packet** sent over a **topic**, from **publisher nodes** to **subscriber nodes**, with a specific **QoS policy**.

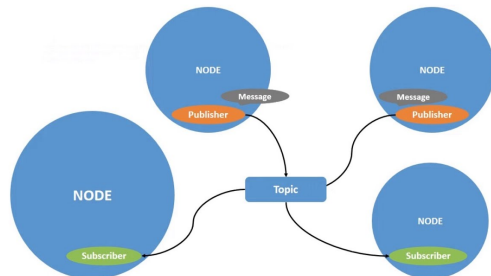


Figure 3: Example of a topic with multiple publisher and subscriber nodes.

Figure 3 is also an example of a simple **node graph**, a pivotal concept in a distributed context.

Interface files

Messages

Interface files format is **derived from the DDS standard**, with data types resolved to machine types according to the platform being used¹.

Message file names end with `.msg`.

Things start very simply...

```
1 int64 data
2
```

Listing 1: Definition of the `std_msgs/msg/Int64` message.

```
1 string data
2
```

Listing 2: Definition of the `std_msgs/msg/String` message.

¹[About ROS 2 interfaces](#) (ROS 2 Jazzy docs)

Interface files

Messages

... then escalate quickly!

```
1 std_msgs/Header header
2 # ^ This includes another message type as a sub-structure
3
4 uint32 height
5 uint32 width
6
7 string encoding
8
9 uint8 is_bigendian
10 uint32 step
11 uint8[] data # This specifies a variable-length uint8 array
12
```

Listing 3: Definition of the sensor_msgs/msg/Image nested message.

Interface files

Messages

Special values (*i.e.*, **constants**) or **default values** may be specified.

```
1 int64 MYNUM=1 # Must be of compatible type
2
3 int64 number
4 int64 number_with_default 2
5
```

Listing 4: Definition of an example message with a constant and a default value.

They are not bound to any field and will appear as **special selectable values** in the generated C++/Python (libraries) code.

ROS 2 adds its own **guidelines**², and installed interfaces can be **inspected** with

`ros2 interface show`

²[ros2-examples/interfaces.md](https://ros2-examples.github.io/interfaces.md)

Message topics

Quality of Service

A QoS policy for a **topic** is a data structure with the following attributes.

- **History** (*keep last N or all*)
- **Depth** (queue size N)
- **Reliability** (*best-effort or reliable*, default: reliable)
- **Durability** (publishers resend all messages to "late-joiners")
- Deadline
- Lifespan
- Liveliness
- Lease duration

On the structure of a ROS 2 QoS policy

The full number of attributes of a ROS 2 QoS policy **comes from the DDS world**.

The **DDS** standard defines **22 QoS settings**, many of which have been **mapped directly in ROS**, many more have been **abstracted away by RMW**.

Some of the QoS settings of the **DDS only provide a notification** that a condition is violated without actively enforcing it (e.g., temporal properties such as deadlines), or **have never been implemented** by vendors because of obsolescence.

ROS 2 inevitably inherited some of this complexity, sometimes unnecessarily.

Zenoh simplifies this scenario with a lean implementation:

- a minimal set of **core settings** are exposed (see previous marked ones);
- some have become **protocol features** guaranteed by the implementation.

On ROS QoS profiles

Since ROS 1, client libraries have always provided **QoS profiles for standard scenarios**:

- default (**reliable**, keep last 10, volatile);
- sensor_data for **continuous streams of samples** (**best-effort**, keep last 5, volatile: **if one is lost, the following ones should make it through...**);
- services, parameters, and other ROS internal communications.

ROS QoS profiles have been designed in a past time when ROS was not mature and widespread as it is today. In real scenarios, it is the communication technologies that should be selected, refined, and tuned to guarantee reliability.

Zenoh is solving this by supporting many more real-world networks and use cases than the DDS.

Message topics

Coding hints for publishers and subscribers

- A node **publishes** on a topic with a **Publisher object**.
 - ▶ The **topic name** must be specified.
 - ▶ A **QoS policy** may be specified.
 - ▶ The `publish` method is then used to **send messages**.
- A node **subscribes** to a topic with a **Subscriber object**.
 - ▶ The **topic name** must be specified.
 - ▶ A **QoS policy** may be specified.
 - ▶ A **callback** must be specified, to execute on each incoming message, which is also its only argument.

Message topics

Inspection tools

The command line tool `ros2 topic` can be used to **inspect topics** and related entities. It has a lot of **verbs**, the most important ones are:

- `list` (list all topics)
- `echo` (print messages to the console)
- `pub` (publish messages from the console)
- `hz` (print publishing rate and statistics)
- `info` (print information about a topic)
- `type` (print the message type)

each with many useful options.

Nodes can be inspected with the `ros2 node` command, and its many verbs.

Timers

Types and coding hints

In ROS 2, **timers** can be used to schedule **jobs to be executed periodically**.

There are **two types**, differentiated by their **time source**.

- **GenericTimer** created with `create_timer`: takes the time from the **ROS node clock**.
 - ▶ This defaults to the kernel real time clock but may be **reset** or set using the `/clock` **topic** (e.g., by a **simulator** setting the `use_sim_time` parameter to true).
- **WallTimer** created with `create_wall_timer`: takes the time from the **OS**.
 - ▶ This goes with the kernel real time clock.

Both must be given a **callback to execute on each tick**, taking no arguments.

Roadmap

1 ROS 2 development workflow

2 Message topics

3 Hello, world!

Code examples

Find all course materials on **GitHub** at [ros2-examples](#) (jazzy branch **updated!** Please pull).

The repository is organized as a **ROS 2 workspace** ready to be built, and intended to support [Visual Studio Code](#) as **IDE**. Find all information in [README.md](#).

It is also organized with **Docker containers** in mind, and supports development containers for VS Code (more on this later).

It is based on the [Distributed Unified Architecture](#), developed during my PhD.

Their inner workings are totally transparent, but if you're curious see [dua-template.md](#).

Suggestion: clone it and checkout your own branch locally, to be still able to get and merge updates from the remote.

Example

Topic pub/sub

Now go have a look at the first example packages:

[ros2-examples/src/cpp/topic_pubsub_cpp](#)

[ros2-examples/src/cpp/advanced/smp_example](#)

[ros2-examples/src/cpp/custom_topic_cpp](#)

- Install ROS 2 on a platform of your choice.
- Run the [demo nodes](#).
- Inspect the demo topics.
- Interact with the demo nodes from the command line.
- Clone [ros2-examples](#) as suggested and rebuild the first example packages.
- Listen to the `/rosout` topic from the command line, then run other nodes.